[Automation] ITOps daily standup

RD-1735: [AdHoc] It operations automation: business analysis CLOSED

RnD repo: inDriver/itns_daily_standup_automation_bot

Goal

This bot goal is to automate daily standup workflows by:

- **Fetching active Jira tasks** from configured filters
- **Analyzing task quality** using AI (completeness of action plans, current/target states)
- **Sending standup reminders** to Slack with task summaries
- **Providing intelligent feedback** to assignees when tasks need attention
- **Running on schedule** (weekdays at 05:30, skips weekends)

Prerequisites

- **Google Cloud credentials** with Vertex AI enabled
- **Jira account** with API token
- **Slack Bot token** with appropriate permissions

Behavior:

- Triggers at 09:30 every weekday (Asia/Almaty timezone)
- Fetches Jira tasks from filter ID 18686 (configurable)
- Automatically excludes Epic issue types
- Sends standup reminder to Slack #**itns-standups** with active tasks
- Creates a thread for feedback messages
- Analyzes each task with AI (completeness, activity, quality)
- Posts feedback in thread for tasks requiring action (with 👀 emoji)
- **Success message**: If all tasks are good, posts "all clear" message mentioning team lead
- Skips execution on weekends (Saturday & Sunday)

```

┌──────────────
```

```
┌──────────────
```

LLM report

```
{
  "decision": "✅ No action required",
  "findings": [
    "The task is 'In Progress' and shows recent activity, with the last comment posted 2 hours ago.",
    "The 'Current state', 'Target state', and 'Action plan' sections are all clear, complete, and well-defined.",
    "The assignee is actively working on the task, troubleshooting issues, and providing regular updates in the comments."
  ],
  "required_actions": [],
  "is_action_required": false,
  "message_to_assignee": null
}
```

💬 Slack Integration

- Sends formatted standup reminders with active tasks grouped by assignee
- Mentions team members using Slack user IDs
- Posts AI-generated feedback as threaded messages
- Parses Slack thread links for context
- Supports channel-wide notifications

🎯 Jira Integration

- Fetches tasks from Jira filters
- Extracts task metadata, descriptions, comments, and changelogs
- Tracks status changes and activity timestamps
- Posts comments back to Jira issues
- Supports nested lists, bullet points, and formatted content
- Handles mentions, inline code, and special formatting

🧠 LLM Integration

- Uses Google Gemini (gemini-2.0-flash-lite) via Vertex AI
- Temperature: 0.1 for consistent, focused responses
- Structured output with Pydantic models
- Automatic retry with exponential backoff

We use structured output to generate strict-formatted reports

```python
from typing import List, Optional
from google.genai import Client
from pydantic import BaseModel, Field

class JiraTaskAnalysis(BaseModel):
    """Basic Jira task analysis"""

    decision: str = Field(..., description="Decision for the task")
    findings: List[str] = Field(default_factory=list, description="Findings")
    required_actions: List[str] = Field(
        default_factory=list, description="Required actions"
    )
    is_action_required: bool = Field(
        default=False, description="Whether action is required"
    )
    message_to_assignee: Optional[str] = Field(
        default=None, description="Ready-to-send message to the assignee in Russian"
    )
```

```python

api_client = Client(vertexai=True)

response = api_client.models.generate_content(
    model=model,
    contents=dialog_contents,
    config=types.GenerateContentConfig(
        system_instruction=system_prompt,
        temperature=0.1,
        # top_p=0.95,
        # top_k=20,
        response_mime_type="application/json",
        response_schema=JiraTaskAnalysis,
    ),
)
```